



WaterAlert

Cahier des Charges

Solution de Monitoring et de Gestion des Fuites d'Eau

Statut	Conforme à l'implémentation réelle
Langage	Python 3.x
Plateforme	Telegram Bot + Streamlit + FastAPI

1. Présentation du Projet

WaterAlert est une solution professionnelle de signalement et de monitoring des fuites d'eau. Elle permet aux citoyens de signaler des fuites géolocalisées accompagnées de photos via un bot Telegram, tandis qu'un tableau de bord analytique propulsé par l'intelligence artificielle permet d'analyser les données en temps réel et de faciliter les décisions d'intervention.

La solution repose sur trois composants complémentaires et indépendants : un bot Telegram pour le signalement citoyen, un tableau de bord Streamlit pour la visualisation et l'analyse des données, et une API REST FastAPI pour l'interopérabilité avec des systèmes externes.

2. Problématique

Le gaspillage d'eau potable causé par les fuites non signalées entraîne des pertes économiques importantes et contribue à la pénurie d'eau dans les zones urbaines. Les systèmes de signalement traditionnels (appels téléphoniques, formulaires web) sont souvent lents, peu accessibles et produisent des données inexploitable.

WaterAlert répond à ce problème en proposant un canal de signalement rapide et accessible via Telegram, combiné à une analyse intelligente par vision artificielle pour qualifier automatiquement la gravité des fuites et permettre une priorisation efficace des interventions techniques.

3. Objectifs

3.1 Objectifs principaux

- Mettre en place un système de signalement des fuites d'eau accessible à tous les citoyens.
- Analyser automatiquement la gravité de chaque fuite grâce à la vision artificielle (Google Gemini).
- Centraliser les données de signalement dans une base de données structurée.
- Faciliter la prise de décision pour les interventions techniques via un tableau de bord analytique.
- Géolocaliser les signalements et identifier les zones critiques sur une carte de chaleur.

3.2 Objectifs secondaires

- Exposer les données de signalement via une API REST pour permettre l'intégration avec des outils tiers.
- Permettre l'exportation des données et des rapports d'intervention en CSV et PDF.
- Garantir la robustesse du système grâce à un mode de simulation automatique en cas d'indisponibilité de l'IA.

- Sécuriser les credentials et les données sensibles via variables d'environnement.

4. Utilisateurs et Profils

Profil	Canal d'accès	Actions disponibles
Citoyen	Bot Telegram	Signaler une fuite, envoyer une photo, partager sa géolocalisation, suivre les commandes /start /help /status /about /privacy /contact
Analyste / Opérateur	Dashboard Streamlit	Consulter les KPIs, explorer les graphiques de tendances, visualiser la carte de chaleur, exporter les données en CSV ou PDF
Développeur / Système tiers	API REST (FastAPI)	Récupérer les signalements de fuites via l'endpoint REST pour intégration dans d'autres outils

5. Fonctionnalités Implémentées

5.1 Bot Telegram — Signalement citoyen

- Workflow de signalement guidé : description, envoi de photo, géolocalisation.
- Commandes standard : /start, /help, /status, /about, /privacy, /contact.
- Menu de commandes cliquable et indicateurs de saisie ('typing...') pour une expérience premium.
- Détection hybride de la gravité : combinaison du choix citoyen et de l'analyse IA (Google Gemini Vision).
- Mode simulation automatique si la clé Gemini est absente ou invalide.

5.2 Intelligence Artificielle — Analyse visuelle

- Intégration de Google Gemini Vision pour l'analyse automatique des photos de fuites.
- Estimation de la gravité de la fuite (faible / modérée / critique) à partir de l'image.
- Système de double vérification combinant l'évaluation citoyenne et le diagnostic de l'IA.
- Dégradation gracieuse du service en mode simulation si l'API IA est indisponible.

5.3 Tableau de bord analytique — Dashboard Streamlit

- Visualisation en temps réel des indicateurs clés de performance (KPIs).
- Graphiques Plotly interactifs : tendances, répartition par gravité, performance par zone.
- Carte de chaleur (heatmap) géographique des zones critiques, pondérée par la sévérité des fuites.
- Filtrage des données et export en CSV ou en rapport PDF d'intervention.

5.4 Intelligence géographique — Localisation

- Géocoding des adresses via la bibliothèque geopy pour convertir les localisations en coordonnées.
- Affichage cartographique interactif des signalements avec précision géographique.
- Identification automatique des zones à forte concentration de fuites.

5.5 API REST — Accès programmatique

- Endpoint FastAPI pour la récupération structurée des signalements de fuites.
- Documentation automatique de l'API (Swagger UI intégré à FastAPI).
- Interopérabilité avec des systèmes de gestion municipaux ou applications mobiles tierces.

5.6 Infrastructure et opérations

- Base de données SQLite pour la persistance locale des signalements (historique + tracking).
- Script de diagnostic `verify_setup.py` pour valider l'installation complète.
- Gestion sécurisée des credentials via fichier `.env` et protection `.gitignore`.
- Environnement virtuel Python (venv) recommandé et documenté pour l'isolation des dépendances.

6. Contraintes Techniques

Contrainte	Choix retenu	Justification
Langage de programmation	Python 3.x	Écosystème riche, bibliothèques IA et data disponibles
Interface citoyenne	Bot Telegram (python-telegram-bot)	Accessible sans installation, API gratuite et robuste
Interface analytique	Streamlit	Développement rapide de dashboards Python, déploiement simplifié
API REST	FastAPI	Hautes performances, documentation Swagger automatique
Base de données	SQLite	Déploiement sans serveur, adapté à un usage mono-instance
IA / Vision	Google Gemini Vision API	Analyse d'images state-of-the-art, intégration Python simple
Géocoding	geopy	Bibliothèque Python open source, multi-fournisseurs
Environnement	venv Python obligatoire	Isolation des dépendances, reproductibilité
Secrets / Sécurité	Fichier <code>.env</code> + <code>.gitignore</code>	Protection des tokens Telegram et clés API

7. Architecture du Projet

Le projet adopte une architecture modulaire avec séparation stricte des responsabilités. Chaque composant peut être démarré, testé et maintenu indépendamment des autres.

Module	Chemin	Rôle
Bot Telegram	src/bot/	Signalement citoyen, intégration Gemini Vision
Dashboard	src/dashboard/	Interface Streamlit, graphiques Plotly, assets CSS
API REST	src/api/	Endpoints FastAPI d'accès aux données
Base de données	src/database/	Couche d'abstraction SQLite (historique & tracking)
Utilitaires	src/utils/	IA (Gemini), géocoding (geopy), génération PDF
Données	data/	Fichiers SQLite persistants
Point d'entrée	streamlit_app.py	Lancement principal pour le déploiement
Diagnostic	verify_setup.py	Validation de l'installation complète

7.1 Lancement des composants

Chaque composant dispose de sa propre commande de démarrage :

- Bot Telegram : `python src/bot/telegram_bot.py`
- Dashboard : `python -m streamlit run streamlit_app.py`
- API REST : `python src/api/main.py`
- Diagnostic : `python verify_setup.py`

8. Résultats Attendus et Livrables

8.1 Résultats attendus

Un système fonctionnel, déployable et documenté permettant de couvrir l'intégralité du cycle de vie d'un signalement de fuite d'eau : de la détection citoyenne via Telegram jusqu'à la visualisation analytique sur le dashboard, en passant par l'analyse automatique par IA et l'exposition des données via API.

8.2 Livrables

- Code source structuré et modulaire avec séparation claire des composants.
- Bot Telegram opérationnel avec commandes standard et analyse IA intégrée.
- Dashboard Streamlit avec KPIs, graphiques Plotly et carte de chaleur.
- API REST FastAPI documentée (Swagger auto-généré).
- Base de données SQLite avec schéma versionné.
- Script de diagnostic `verify_setup.py`.
- Documentation technique complète (README.md).

- Fichier requirements.txt listant toutes les dépendances.

9. Configuration et Variables d'Environnement

Le fichier .env à la racine du projet doit contenir les variables suivantes :

Variable	Description	Obligatoire
TELEGRAM_BOT_TOKEN	Token BotFather pour le bot Telegram	Oui
GEMINI_API_KEY	Clé Google AI Studio pour Gemini Vision	Non (mode simulation si absent)

En l'absence de GEMINI_API_KEY, le système bascule automatiquement en mode simulation pour maintenir la continuité du service sans interruption.